

Data Lab 实验报告

姓名： 杨一帆
学号： 25803060020

一、实验目的

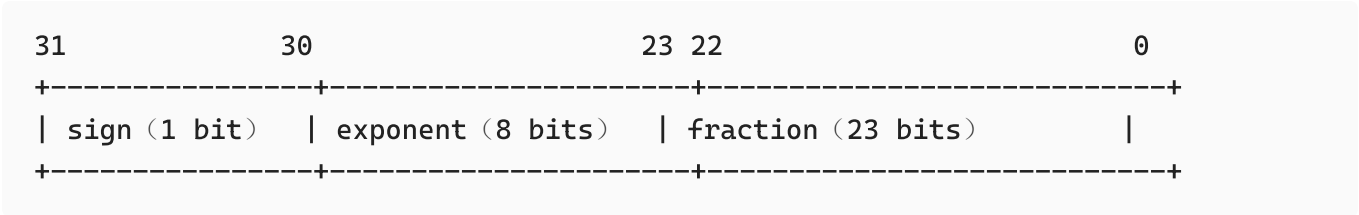
本实验要求在严格限制运算符和控制结构的条件下完成一组整数与浮点数操作。实验的主要目标如下：

- 1. 熟悉 32 位二进制补码的表示以及算术右移、逻辑右移的区别；
- 2. 掌握使用掩码、移位和布尔运算提取、清除、复制及交换位段的方法；
- 3. 理解有符号整数加法、乘法产生溢出的条件，并在不能使用更宽整数类型时判断溢出方向；
- 4. 理解 IEEE 754 单精度浮点数的符号位、阶码和尾数结构；
- 5. 掌握“向最接近值舍入，恰好居中时向偶数舍入”的实现方法。

二、实验基础

整数部分采用 32 位补码模型，最高位为符号位。负数算术右移时高位补 1，正数算术右移时高位补 0。题目允许的主要运算符为 `!`、`~`、`&`、`^`、`|`、`+`、`<<` 和 `>>`，因此减法通常写为 `x + ~y + 1`，取负数写为 `~x + 1`。

IEEE 754 单精度浮点数的位布局为：



当阶码为 1~254 时，有效数含隐藏的最高位 1；阶码为 0 时表示零或非规格化数，不含隐藏位；阶码为 255 时表示无穷大或 NaN。

三、各函数的实现思路

P1 signMask

题目大意

构造一个只有最高有效位为 1、其余 31 位均为 0 的掩码，即返回 `0x80000000`。本题最多使用 2 个允许的位运算。

实现思路

目标是只将 32 位整数的最高位置 1。常数 1 的最低位为 1，将它左移 31 位即可得到：

```
00000000 00000000 00000000 00000001
          左移 31 位
100000000 000000000 000000000 000000000
```

因此直接返回 `1 << 31`，结果即为 `0x80000000`。该方法只需要一次移位。

实现代码

```
int signMask(void) {
    return 1 << 31;
}
```

P2 bitXor

题目大意

只使用按位取反 `~` 和按位与 `&` 实现 `x ^ y`。当两个数的对应位不同时，结果位应为 1。

实现思路

异或的含义是“两位不同则为 1”，可以拆为两种互斥情况：

$$(x \ \& \ \sim y) \mid (\sim x \ \& \ y)$$

题目只允许使用 `~` 和 `&`，所以再利用德摩根律 $a \mid b = \sim(\sim a \ \& \ \sim b)$ ，将按位或改写为：

$$\sim(\sim(x \ \& \ \sim y) \ \& \ \sim(\sim x \ \& \ y))$$

这样便在不直接使用 `^` 和 `|` 的情况下实现了异或。

实现代码

```
int bitXor(int x, int y) {
    return \sim(\sim(x \ \& \ \sim y) \ \& \ \sim(\sim x \ \& \ y));
}
```

P3 negativePart

题目大意

当 `x` 为负数时返回 `-x`，当 `x` 为非负数时返回 `0`。实现中不能使用条件语句。

实现思路

先用 `x >> 31` 取得符号掩码：非负数得到全 `0`，负数得到全 `1`。随后用补码规则 `~x + 1` 计算 `-x`，再与符号掩码相与：

- 当 `x >= 0` 时，掩码为 `0x00000000`，结果被清零；
- 当 `x < 0` 时，掩码为 `0xffffffff`，保留 `-x` 的所有位。

因此表达式 `(~x + 1) & (x >> 31)` 能在不使用条件语句的情况下完成选择。

实现代码

```
int negativePart(int x) {
    int mask = x >> 31;
    return (~x + 1) & mask;
}
```

P4 copyByteWithin

题目大意

把整数 `x` 的第 `src` 个字节复制到第 `dst` 个字节，同时保持其余字节不变。字节编号从最低有效字节 `0` 到最高有效字节 `3`。

实现思路

字节编号从最低字节开始，每个字节占 `8` 位，因此源字节和目标字节的起始位分别为 `src << 3` 与 `dst << 3`。

实现分三步：

1. 将 `x` 右移到源字节所在位置，并用 `0xff` 取出该字节；
2. 将 `0xff` 左移到目标位置，取反后与 `x` 相与，以清空原目标字节；
3. 将源字节左移到目标位置，再与清空目标字节后的结果按位或。

源字节先经过 `& 0xff`，所以即使 `x` 为负数、右移时高位补 `1`，也不会污染提取结果。当 `src == dst` 时，函数会先清除再写回同一个字节，结果仍等于原值。

实现代码

```
int copyByteWithin(int x, int src, int dst) {
    int srcShift = src << 3;
```

```

int dstShift = dst << 3;
int byte = (x >> srcShift) & 0xFF;
int dstMask = 0xFF << dstShift;
int cleared = x & ~dstMask;
return cleared | (byte << dstShift);
}

```

P5 logicalShift

题目大意

在机器只提供有符号算术右移的情况下，实现把 `x` 逻辑右移 `n` 位。逻辑右移要求左侧始终补 0，且 $0 \leq n \leq 31$ 。

实现思路

C 语言中对有符号数执行右移通常是算术右移，负数右移时左侧会补 1。函数先执行 `x >> n`，然后构造一个低 `32-n` 位为 1、高 `n` 位为 0 的掩码，清除算术右移补入的符号位。

`1 << 31` 得到最高位掩码。对它算术右移 `n` 位后再左移一位，可以得到高 `n` 位为 1 的区域，取反后便得到所需掩码：

```
mask = ~(((1 << 31) >> n) << 1);
```

最终返回 `(x >> n) & mask`。当 `n == 0` 时掩码为全 1；当 `n == 31` 时掩码只保留最低一位。

实现代码

```

int logicalShift(int x, int n) {
    int highBit = 1 << 31;
    int mask = ~((highBit >> n) << 1);
    return (x >> n) & mask;
}

```

P6 swapNibblePairs

题目大意

交换 `x` 中每个字节的高 4 位和低 4 位。例如，一个字节 `0xAB` 交换后应变为 `0xBA`。

实现思路

一个字节由两个 4 位半字节组成。函数先从 `0x0f` 扩展出掩码 `0x0f0f0f0f`，它恰好覆盖每个字节的低 4 位。

- $(x \& \text{mask}) \ll 4$ ：取出所有低半字节并移动到高半字节位置；
- $(x \gg 4) \& \text{mask}$ ：将所有高半字节移动到低半字节位置，并用掩码清除算术右移可能补入的符号位；
- 将两部分按位或，得到交换后的结果。

所有字节同时处理，不需要循环逐字节操作。

实现代码

```
int swapNibblePairs(int x) {
    int mask = 0x0F;
    mask = mask | (mask << 8);
    mask = mask | (mask << 16);
    return ((x & mask) << 4) | ((x >> 4) & mask);
}
```

P7 secondLowestZeroBit

题目大意

返回一个只标记 x 中第二低的 0 位的掩码。如果第二个 0 位不存在，则返回 0。

实现思路

表达式 $\sim x \& (x + 1)$ 可以定位 x 中最低的 0 位。原因是 $x + 1$ 会把最低 0 位变成 1，并把它右侧连续的 1 变成 0；再与 $\sim x$ 相与后，只剩这个最低 0 位。

函数先计算：

```
first = ~x & (x + 1);
filled = x | first;
```

这相当于把最低的 0 位临时填成 1。然后对 `filled` 再使用一次相同方法：

```
~filled & (filled + 1)
```

所得位置就是原数中第二低的 0 位。如果不存在第二个 0 位，例如 $x == -1$ 或 $x == 0x7fffffff$ ，补码回绕会使结果自然变为 0。

实现代码

```
int secondLowestZeroBit(int x) {
    int first = ~x & (x + 1);
```

```
int filled = x | first;
return ~filled & (filled + 1);
}
```

P8 oddParity

题目大意

计算 `x` 的奇校验位：若 `x` 的二进制表示中 1 的数量为偶数则返回 1，为奇数则返回 0。

实现思路

该函数返回需要附加的奇校验位：当 `x` 中 1 的个数为偶数时返回 1，否则返回 0，使“原数据加校验位”后的 1 总数为奇数。

实现采用异或折叠：

```
x ^= x >> 16;
x ^= x >> 8;
x ^= x >> 4;
x ^= x >> 2;
x ^= x >> 1;
```

每执行一次，统计范围都会扩大一倍。最终最低位等于原 32 位中所有位的异或结果，即原数中 1 的个数的奇偶性。若最低位为 0，说明 1 的个数为偶数，此时 `!(x & 1)` 返回 1；反之返回 0。

实现代码

```
int oddParity(int x) {
    x = x ^ (x >> 16);
    x = x ^ (x >> 8);
    x = x ^ (x >> 4);
    x = x ^ (x >> 2);
    x = x ^ (x >> 1);
    return !(x & 1);
}
```

P9 rotateRightBits

题目大意

把 32 位整数 `x` 循环右移 `n` 位：从最低位移出的内容要重新进入最高位。题目只保证 `n` 非负，因此还要处理大于等于 32 的位数。

实现思路

循环右移由“逻辑右移得到的低位部分”和“从右端绕回高端的部分”组成。由于题目只保证 $n \geq 0$ ，代码先用 $n \& 31$ 将位数归一化到 0~31：

```
shift = n & 31;
leftShift = (32 - shift) & 31;
```

其中减法按允许的补码形式写成 $32 + (\sim\text{shift} + 1)$ 。右侧部分复用逻辑右移的掩码构造方法；左侧部分为 $x \ll \text{leftShift}$ 。最后把两部分按位或即可。

对 32 取模还有一个重要作用：当 $\text{shift} == 0$ 时， leftShift 也为 0，避免执行非法的 32 位移位，最终结果仍为 x 。

实现代码

```
int rotateRightBits(int x, int n) {
    int shift = n & 31;
    int leftShift = (32 + (~shift + 1)) & 31;
    int highBit = 1 << 31;
    int mask = ~((highBit >> shift) << 1);
    int right = (x >> shift) & mask;
    int left = x << leftShift;
    return right | left;
}
```

P10 roundEvenPow2

题目大意

把非负整数 x 舍入到最接近的 2^n 的倍数；若 x 恰好位于两个倍数的中间，则选择除以 2^n 后商为偶数的那个倍数。

实现思路

将非负整数 x 舍入到最接近的 2^n 倍数，可将它分为商和余数：

$$x = q \times 2^n + \text{rem}$$

代码中 $q = x \gg n$ ， $\text{rem} = x \& ((1 \ll n) - 1)$ ，中点为 $\text{half} = 1 \ll (n - 1)$ 。

舍入规则为：

- $\text{rem} < \text{half}$ ：向下舍入；

- `rem > half`：向上舍入；
- `rem == half`：若 `q` 为奇数则向上，使新商变为偶数；若 `q` 已为偶数则向下。

代码用 `half - rem` 的符号位判断 `rem > half`，用 `!(rem ^ half)` 判断二者相等，最后得到 `roundUp`，并返回 `(q + roundUp) << n`。这正是整数版本的“舍入到最近偶数”。

实现代码

```
int roundEvenPow2(int x, int n) {
    int mask = (1 << n) + ~0;
    int rem = x & mask;
    int half = 1 << (n + ~0);
    int q = x >> n;

    int greater = ((half + ~rem + 1) >> 31) & 1;
    int equal = !(rem ^ half);
    int roundUp = greater | (equal & (q & 1));

    return (q + roundUp) << n;
}
```

P11 midpointTowardFirst

题目大意

在不发生加法溢出的前提下求 `x` 与 `y` 的数学中点。若中点恰好位于两个整数之间，则选择更靠近第一个参数 `x` 的整数。

实现思路

直接计算 `(x + y) / 2` 可能在 `x + y` 时溢出。恒等式

```
(x & y) + ((x ^ y) >> 1)
```

可以在不先求和的情况下得到两数平均值的向下取整结果 `base`。其中 `x & y` 表示两数共同具有的高位，`(x ^ y) >> 1` 表示差异部分的一半。

如果 `x` 与 `y` 奇偶性不同，即 `(x ^ y) & 1` 为 1，则精确中点位于两个整数之间。`base` 默认选择较小的相邻整数：

- 若 `x < y`，较小整数更靠近第一个参数 `x`，无需调整；
- 若 `x > y`，应选择较大整数，因此给 `base` 加 1。

代码综合符号位与差值符号，在避免比较溢出的情况下得到 `xGreater`，最终只在“中点为半整数且 `x > y`”时加 1。

实现代码

```
int midpointTowardFirst(int x, int y) {
    int base = (x & y) + ((x ^ y) >> 1);
    int sx = x >> 31;
    int sy = y >> 31;
    int sameSign = ~(sx ^ sy);
    int diff = x + ~y + 1;

    int xGreater =
        (((~sx) & sy) |
         (sameSign & ~(diff >> 31) & !!diff)) & 1;

    return base + (((x ^ y) & 1) & xGreater);
}
```

P12 isBetweenEitherOrder

题目大意

判断 `x` 是否位于以 `a`、`b` 为端点的闭区间内。两个端点可以按任意顺序给出，位于端点上也应返回 1。

实现思路

要判断 `x` 是否位于端点 `a`、`b` 之间，需要同时支持 `a <= b` 和 `b <= a` 两种顺序：

$$(x \geq a \text{ 且 } x \leq b) \text{ 或 } (x \geq b \text{ 且 } x \leq a)$$

难点在于直接通过 `x - a` 的符号判断大小可能因减法溢出而出错。以判断 `x < a` 为例，先令 `d = x - a`，再使用：

$$(d \wedge ((x \wedge a) \& (x \wedge d))) \gg 31$$

当 `x` 与 `a` 异号时，比较结果由 `x` 的符号决定；同号时则由差值符号决定。该表达式把两种情况统一起来。代码分别计算 `x < a`、`b < x`、`x < b` 和 `a < x`，再组合两个闭区间条件。使用“严格小于”的反值可自然包含端点相等的情况。

实现代码

```

int isBetweenEitherOrder(int x, int a, int b) {
    int dxa = x + ~a + 1; /* x-a */
    int dbx = b + ~x + 1; /* b-x */
    int dxb = x + ~b + 1; /* x-b */
    int dax = a + ~x + 1; /* a-x */

    /* lxa 表示 x<a, 其他变量同理 */
    int lxa = ((dxa ^ ((x ^ a) & (x ^ dxa))) >> 31) & 1;
    int lbx = ((dbx ^ ((b ^ x) & (b ^ dbx))) >> 31) & 1;
    int lxb = ((dxb ^ ((x ^ b) & (x ^ dxb))) >> 31) & 1;
    int lax = ((dax ^ ((a ^ x) & (a ^ dax))) >> 31) & 1;

    return ((!lxa) & (!lbx)) | ((!lxb) & (!lax));
}

```

P13 mul5Sat

题目大意

计算 $x * 5$ 。若精确结果超出 32 位有符号整数范围，正溢出时返回 `INT_MAX`，负溢出时返回 `INT_MIN`。

实现思路

乘以 5 被拆成 $x + (x \ll 2)$ 。溢出可能出现在两个阶段：

1. $x \ll 2$ 本身丢失高位。若将结果算术右移 2 位不能恢复 x ，说明移位溢出；
2. x 与 `four` 相加时溢出。两个加数同号但结果异号，是补码加法溢出的标准判据。

只要任一阶段溢出，就应饱和到边界。`~(1 << 31)` 构造 `INT_MAX`，再与 x 的全 0/全 1 符号掩码异或：正数得到 `INT_MAX`，负数得到 `INT_MIN`。

最后将布尔量 `overflow` 扩展成全位掩码，用

```
(mask & saturated) | (~mask & product)
```

在饱和值和正常乘积之间完成无分支选择。

实现代码

```

int mul5Sat(int x) {
    int four = x << 2;
    int product = x + four;
}

```

```

int shiftOverflow = !!(four >> 2) ^ x);
int addOverflow =
    ((~(x ^ four) & (x ^ product)) >> 31) & 1;
int overflow = shiftOverflow | addOverflow;

int sign = x >> 31;
int intMax = ~(1 << 31);
int saturated = intMax ^ sign;

int mask = ~overflow + 1;
return (mask & saturated) | (~mask & product);
}

```

P14 classifyAdd3

题目大意

判断三个有符号整数的精确数学和 $x + y + z$ 是否越界：大于 `INT_MAX` 返回 1，小于 `INT_MIN` 返回 -1，否则返回 0，并且不能借助更宽的整数类型。

实现思路

函数要判断精确数学和 $x + y + z$ 是否超出 32 位有符号范围，但不能使用更宽的整数类型。代码分两次加法：先求 $\text{sumXY} = x + y$ ，再求 $\text{sumXYZ} = \text{sumXY} + z$ 。

对每次加法，都用“两个加数同号、结果与加数异号”检测溢出：

```
~(a ^ b) & (a ^ sum)
```

其最高位为 1 时发生溢出。随后根据加数符号把正溢出编码为 1、负溢出编码为 -1。

两次溢出方向相加就是最终分类结果。若第一次正溢出，而加入负数后精确和重新回到范围内，第二次会产生方向相反的负溢出，二者相加为 0；负溢出后的抵消情况同理。如果没有被抵消，则保留 1 或 -1。这一方法借助 32 位回绕和溢出方向抵消，在不使用 64 位整数的情况下判断三数的精确和。

实现代码

```

int classifyAdd3(int x, int y, int z) {
    int sumXY = x + y;
    int sumXYZ = sumXY + z;

    int overflow1 =
        (~(x ^ y) & (x ^ sumXY)) >> 31;
}

```

```

int direction1 =
    overflow1 & ((x >> 31) | 1);

int overflow2 =
    (~(sumXY ^ z) & (sumXY ^ sumXYZ)) >> 31;
int direction2 =
    overflow2 & ((sumXY >> 31) | 1);

return direction1 + direction2;
}

```

P15 floatScaleThreeHalves

题目大意

把参数 `uf` 当作 IEEE 754 单精度浮点数的位模式，返回 $f * 3/2$ 的位模式。结果使用向最近偶数舍入；NaN 原样返回，并保留零的符号。

实现思路

函数将浮点数按位解释后计算 $f \times 3/2$ 。首先分离符号位、阶码和尾数。NaN 与无穷大的阶码均为 `0xff`，按题意原样返回。

1. 非规格化数

非规格化数的值可写为 $\text{frac} \times 2^{-149}$ ，所以只需把整数尾数乘 3 再除以 2。代码令 `product = frac + (frac << 1)`，再右移一位得到商。若被除掉的最低位为 1，说明结果恰在两个可表示数之间；此时仅当商为奇数时加 1，使结果最低位为偶数。

当舍入后的 `q` 达到 `0x800000` 以上时，它的位模式会自然进入阶码为 1 的最小规格化区间，无需单独拼接阶码。

2. 规格化数

规格化数先补出隐藏位：

```
sig = frac | 0x00800000;
```

然后计算 `product = 3 × sig`。若乘积达到 2^{25} ，则需要除以 4 并把阶码加 1；否则只需除以 2，阶码不变。两种分支都检查余数，并按“距离更近优先、正好一半时保留偶数”进行舍入。

舍入可能使有效数从 `1.111...` 进位为 `10.000...`，此时需要再次右移有效数并增加阶码。若阶码最终达到 `0xff`，结果变为相应符号的无穷大；否则重新拼接符号位、阶码和尾数。

实现代码

```

unsigned floatScaleThreeHalves(unsigned uf) {
    unsigned sign = uf & 0x80000000u;
    unsigned exp = (uf >> 23) & 0xffu;
    unsigned frac = uf & 0x007fffffu;
    unsigned sig, product, q, rem;

    /* NaN 和无穷大均原样返回 */
    if (exp == 0xffu)
        return uf;

    /* 非规格化数: value = frac * 2-149 */
    if (exp == 0) {
        product = frac + (frac << 1);
        q = product >> 1;

        /* product/2 恰好为半整数时, 舍入到偶数 */
        if ((product & 1u) && (q & 1u))
            q = q + 1;

        /* q 达到 0x800000 时会自然编码成规格化数 */
        return sign | q;
    }

    sig = frac | 0x00800000u;
    product = sig + (sig << 1);

    /* 有效数超出当前规格化区间时提升指数 */
    if (product & 0x02000000u) {
        q = product >> 2;
        rem = product & 3u;

        if ((rem > 2u) || ((rem == 2u) && (q & 1u)))
            q = q + 1;

        exp = exp + 1;
    } else {
        q = product >> 1;

        if ((product & 1u) && (q & 1u))
            q = q + 1;

        /* 舍入可能令有效数产生进位 */
        if (q & 0x01000000u) {
            q = q >> 1;
            exp = exp + 1;
        }
    }
}

```

```

}

if (exp >= 0xffu)
    return sign | 0x7f800000u;

return sign | (exp << 23) | (q & 0x007fffffu);
}

```

P16 floatRoundEven

题目大意

将 `uf` 所表示的单精度浮点数舍入到最接近的整数，并返回该整数的单精度浮点位模式。恰好居中时舍入到偶数；NaN 和无穷大原样返回，舍入成零时保留输入符号。

实现思路

该函数把浮点数舍入为最接近的整数，但返回值仍是该整数对应的单精度浮点位模式。

根据阶码分区处理：

- `exp == 0xff`：NaN 或无穷大，原样返回；
- `exp < 126`：绝对值小于 0.5，结果为带原符号的零；
- `exp == 126`：绝对值位于 $[0.5, 1)$ 。精确的 0.5 舍入到偶数 0，其余值舍入到 1；
- `exp >= 150`：无偏指数至少为 23，浮点数已没有小数位，原样返回；
- 其余情况：需要清除一部分尾数位并判断是否进位。

在最后一种情况下，小数部分占 $150 - \text{exp}$ 位。代码构造低位掩码得到被舍弃部分 `rem`，并计算中点 `half`。若 `rem > half`，或者 `rem == half` 且保留部分的最低位为 1，就向上增加一个保留单位。由于 IEEE 754 采用符号—数值形式，这个“增加绝对值有效数”的操作对正负数都能实现正确的就近舍入。若结果为零，代码只返回原符号位，从而保留 `+0` 或 `-0`。

实现代码

```

unsigned floatRoundEven(unsigned uf) {
    unsigned sign = uf & 0x80000000u;
    unsigned exp = (uf >> 23) & 0xffu;
    unsigned frac = uf & 0x007fffffu;
    unsigned shift, mask, half;
    unsigned sig, rem, truncated, lsb;

    /* NaN 或无穷大 */
    if (exp == 0xffu)
        return uf;
}

```

```

/* |f| < 0.5, 结果为带原符号的零 */
if (exp < 126u)
    return sign;

/* 0.5 <= |f| < 1 */
if (exp == 126u) {
    if (frac == 0)
        return sign;
    return sign | 0x3f800000u;
}

/* 无偏指数至少为 23, 已经没有小数部分 */
if (exp >= 150u)
    return uf;

shift = 150u - exp;
mask = (1u << shift) - 1u;
half = 1u << (shift - 1u);

sig = frac | 0x00800000u;
rem = frac & mask;
truncated = uf & ~mask;
lsb = (sig >> shift) & 1u;

if ((rem > half) || ((rem == half) && lsb))
    truncated = truncated + (1u << shift);

return truncated;
}

```

P17 float_i2f

题目大意

实现 C 表达式 `(float)x` 的位级等价转换：输入是 32 位有符号整数，返回对应单精度浮点数的无符号位模式，并按 IEEE 754 规则舍入。

实现思路

该函数实现从 32 位有符号整数到单精度浮点位模式的转换。

1. `x == 0` 时直接返回全 0；
2. 保存符号位，并在 `unsigned` 域中用 `~mag + 1` 求绝对值。这样即使输入为 `INT_MIN` 也不会出现有符号取负溢出；

3. 从第 31 位向下查找最高的 1，其位置 e 就是无偏指数，故阶码为 $e + 127$ ；
4. 若 $e \leq 23$ ，整数的所有有效位都能放入尾数，只需左移对齐并去掉隐藏位；
5. 若 $e > 23$ ，右移保留最高 24 个有效位，并依据被丢弃部分进行 ties-to-even 舍入。

当保留的 24 位全为 1 且发生进位时，会得到 `0x01000000`，即有效数从 `1.111...` 变为 `10.000...`。此时将有效数右移一位并增加阶码。最后拼接符号位、阶码和 23 位尾数。

实现代码

```
unsigned float_i2f(int x) {
    unsigned ux = x;
    unsigned sign;
    unsigned mag;
    unsigned exp;
    unsigned frac;
    unsigned q;
    unsigned rem;
    unsigned half;
    int e;
    int shift;

    if (x == 0)
        return 0;

    sign = ux & 0x80000000u;
    mag = ux;

    /* 在 unsigned 域中求绝对值，INT_MIN 也不会溢出 */
    if (sign)
        mag = ~mag + 1u;

    /* 找最高的 1 */
    e = 31;
    while (!(mag & (1u << e)))
        e--;

    exp = e + 127;

    if (e <= 23) {
        frac = (mag << (23 - e)) & 0x007fffffu;
    } else {
        shift = e - 23;
        q = mag >> shift;
        rem = mag & ((1u << shift) - 1u);
        half = 1u << (shift - 1);
```

```

    if ((rem > half) || ((rem == half) && (q & 1u)))
        q = q + 1u;

    /* 舍入令 1.111... 进位成 10.000... */
    if (q == 0x01000000u) {
        q = q >> 1;
        exp = exp + 1u;
    }

    frac = q & 0x007fffffu;
}

return sign | (exp << 23) | frac;
}

```

P18 bitCount

题目大意

统计 32 位整数 `x` 的二进制表示中共有多少个 1，并返回 0~32 之间的计数结果。

实现思路

函数采用并行分组求和（parallel bit count），避免逐位循环。

首先由较小常数扩展出三组掩码：

```

mask1 = 0x55555555 // 每 2 位中的低 1 位
mask2 = 0x33333333 // 每 4 位中的低 2 位
mask4 = 0x0f0f0f0f // 每 8 位中的低 4 位

```

然后依次合并计数：

1. 每相邻 2 位相加，得到每 2 位中的 1 的个数；
2. 每两个 2 位组相加，得到每 4 位中的计数；
3. 每两个 4 位组相加，得到每字节中的计数；
4. 再合并相邻字节和相邻 16 位组，得到整个 32 位整数的计数。

32 位中最多有 32 个 1，只需 6 位表示，因此最终用 `x & 0x3f` 取出结果。

实现代码

```

int bitCount(int x) {
    int mask1 = 0x55 | (0x55 << 8);
    int mask2 = 0x33 | (0x33 << 8);
    int mask4 = 0x0f | (0x0f << 8);

    mask1 = mask1 | (mask1 << 16);
    mask2 = mask2 | (mask2 << 16);
    mask4 = mask4 | (mask4 << 16);

    x = (x & mask1) + ((x >> 1) & mask1);
    x = (x & mask2) + ((x >> 2) & mask2);
    x = (x + (x >> 4)) & mask4;
    x = x + (x >> 8);
    x = x + (x >> 16);

    return x & 0x3f;
}

```

P19 bitReverse

题目大意

将一个 32 位整数的全部二进制位逆序，使原来的第 i 位移动到第 $31-i$ 位。

实现思路

位反转通过分治交换完成。代码先构造掩码：

```

m1  = 0x55555555
m2  = 0x33333333
m4  = 0x0f0f0f0f
m8  = 0x00ff00ff
m16 = 0x0000ffff

```

随后按层次交换：

1. 交换每组中的相邻 1 位；
2. 交换相邻 2 位组；
3. 交换相邻 4 位组；
4. 交换相邻字节；
5. 交换高、低 16 位。

每一步都用掩码取出一半，分别向左或向右移动后重新组合。右移分支始终再与掩码相与，因此算术右移补入的符号位不会影响结果。经过五轮交换后，原第 i 位移动到第 $31-i$ 位，完成完

整的 32 位反转。

实现代码

```
int bitReverse(int x) {
    int m16 = 0xff | (0xff << 8);
    int m8 = m16 ^ (m16 << 8);
    int m4 = m8 ^ (m8 << 4);
    int m2 = m4 ^ (m4 << 2);
    int m1 = m2 ^ (m2 << 1);

    x = ((x >> 1) & m1) | ((x & m1) << 1);
    x = ((x >> 2) & m2) | ((x & m2) << 2);
    x = ((x >> 4) & m4) | ((x & m4) << 4);
    x = ((x >> 8) & m8) | ((x & m8) << 8);
    x = (x << 16) | ((x >> 16) & m16);

    return x;
}
```

四、典型边界情况分析

类别	代表输入	预期行为
符号边界	negativePart(INT_MIN)	在 32 位位向量模型中保留 0x80000000
移位边界	logicalShift(x, 0)、logicalShift(x, 31)	分别不移位、只保留原最高位到最低位
循环移位	rotateRightBits(x, 0)、rotateRightBits(x, 32)	均返回原值，且不执行位数为 32 的移位
不存在第二个零位	secondLowestZeroBit(-1)、secondLowestZeroBit(0x7fffffff)	返回 0
恰好居中	roundEvenPow2(10, 2)、roundEvenPow2(14, 2)	分别得到 8 和 16，商均为偶数
平均值溢出风险	midpointTowardFirst(INT_MAX, INT_MAX)	不直接计算两数之和，结果仍为 INT_MAX
区间端点反序	isBetweenEitherOrder(5, 8, 3)	返回 1
饱和乘法	mul5Sat(0x40000000)	正溢出后返回 INT_MAX
浮点特殊值	NaN、 $+\infty$ 、 $-\infty$	浮点函数按题意原样返回或产生对应无穷大

类别	代表输入	预期行为
有符号零	小负数舍入为整数零	floatRoundEven 返回 -0
整数转浮点	0、INT_MIN、超过 24 位精度的整数	分别处理零、无符号绝对值和 ties-to-even 舍入

五、正确性与复杂度说明

整数函数均不使用循环或条件语句，运行时间和额外空间均为常数。浮点函数 P15、P16 只有固定数量的分支，P17 查找最高有效位的循环最多执行 31 次，因此总体仍可视为常数时间，所有函数的额外空间均为 $O(1)$ 。

从实现结构看，代码覆盖了以下关键正确性条件：

- 通过掩码消除算术右移产生的符号扩展；
- 使用补码等价式实现减法、取负与无分支选择；
- 有符号比较和三数求和分类均显式考虑了溢出；
- 所有要求 ties-to-even 的函数都同时判断“被丢弃部分与中点的关系”和“保留结果的奇偶性”；
- 浮点函数分别处理零、非规格化数、规格化数、无穷大和 NaN；
- 舍入引起的有效数进位与阶码溢出均有相应处理。

六、实验结果

```

claude@WIN-M163BUNC03:/mnt/d/Ethel/Sophomore_FALL/ICS/datalab/datalab-yyf$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 13/16 operators
roundEvenPow2: 20/24 operators
midpointTowardFirst: 25/32 operators
isBetweenEitherOrder: 43/48 operators
mul5Sat: 23/30 operators
classifyAdd3: 19/52 operators
floatScaleThreeHalves: 41/60 operators
floatRoundEven: 27/65 operators
float_i2f: 33/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.

```

```
claire@WIN-M163BUNCG03:/mnt/d/Ethel/Sophomore_FALL/ICS/datalab/datalab-yyf$ ./btest
Score  Rating  Errors  Function
1      1        0      signMask
2      2        0      bitXor
3      3        0      negativePart
4      4        0      copyByteWithin
4      4        0      logicalShift
4      4        0      swapNibblePairs
4      4        0      secondLowestZeroBit
5      5        0      oddParity
5      5        0      rotateRightBits
5      5        0      roundEvenPow2
5      5        0      midpointTowardFirst
7      7        0      isBetweenEitherOrder
7      7        0      mul5Sat
7      7        0      classifyAdd3
7      7        0      floatScaleThreeHalves
10     10        0      floatRoundEven
10     10        0      float_i2f
10     10        0      bitCount
10     10        0      bitReverse
Total points: 110/110
```

七、实验总结

本实验的核心并不是把常规 C 语言程序简单改写为位运算，而是从数据的二进制表示出发重新设计算法。整数部分中，掩码负责选择位，移位负责对齐位段，补码性质负责实现取负、比较和溢出判断；浮点部分则要求把一个算术操作拆成符号、有效数、指数和舍入四个相互关联的步骤。

通过本实验，我进一步理解了以下几点：第一，与全0取与，即为删除；与全1取与，即为保留。32bite为四个字节，8bite为一个字节。第二、算术右移与逻辑右移的差异本质上来来自对最高位是否具有符号含义的解释；第三，整数溢出虽然使普通算术直觉失效，但其符号变化仍能用于构造可靠的判据；第四，浮点运算的难点不仅在于指数对齐，还在于精确处理边界、特殊值和舍入进位。位级实现虽然代码较短，但每个表达式都依赖明确的数据表示规律，因此边界分析与验证和算法本身同样重要。